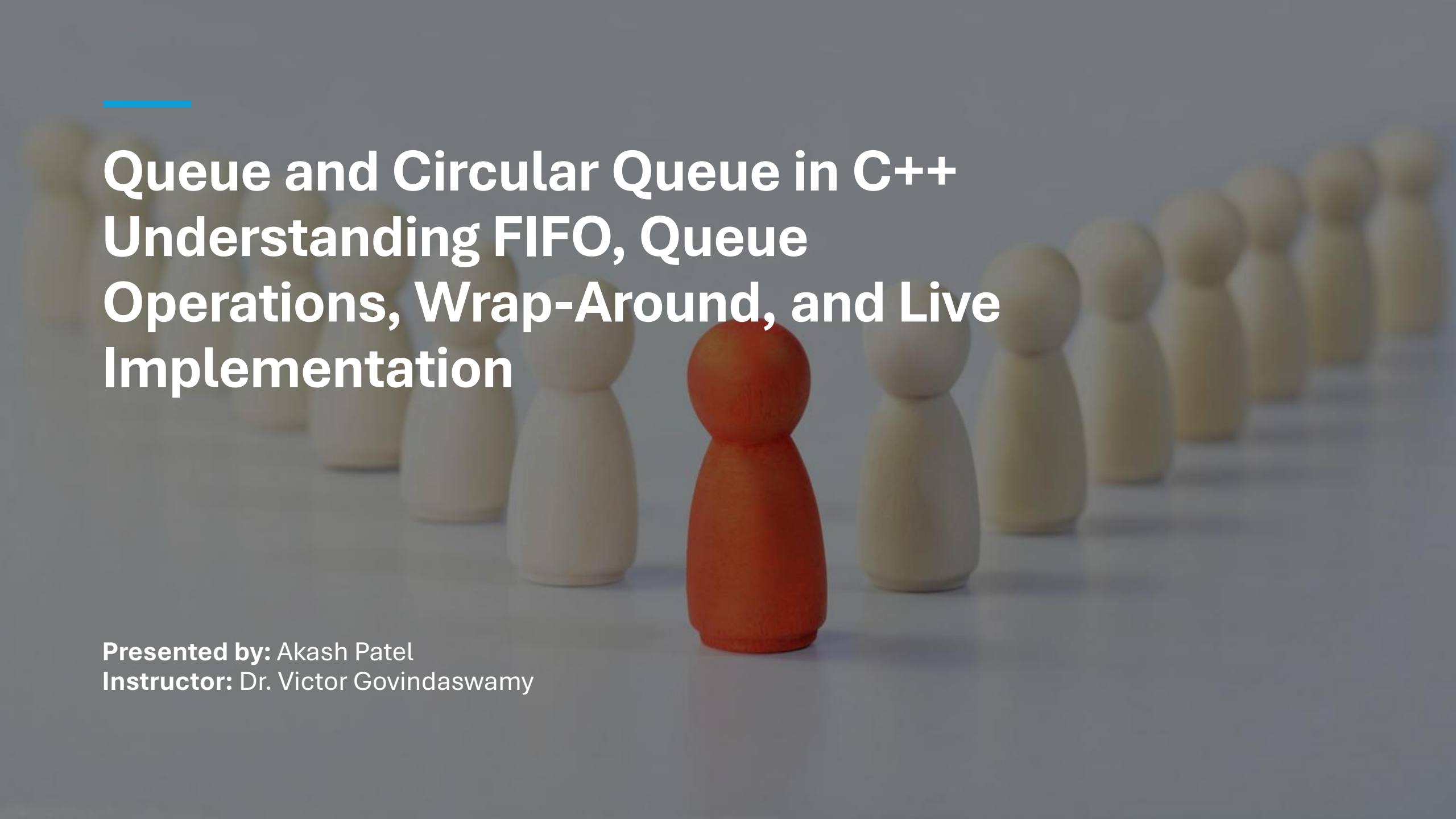


A row of wooden chess pawns is shown, receding into the distance. The pawns are light-colored wood, except for one in the center foreground which is a vibrant red. The background is a soft, out-of-focus grey.

Queue and Circular Queue in C++ Understanding FIFO, Queue Operations, Wrap-Around, and Live Implementation

Presented by: Akash Patel
Instructor: Dr. Victor Govindaswamy



What Is a Queue?

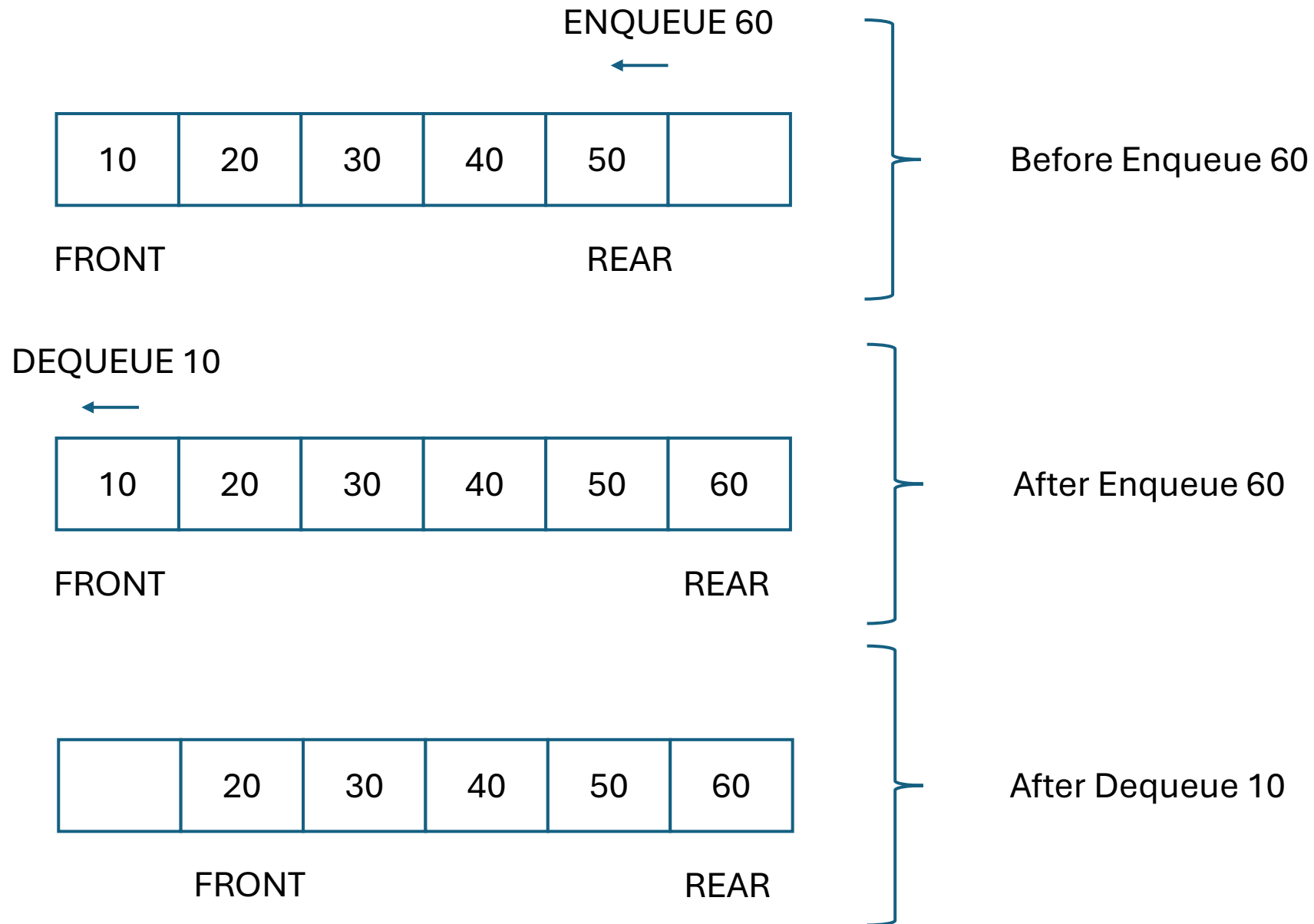
- A **Queue** is a linear data structure where values enter from one side and leave from the other side.
 - The first value that enters the Queue is also the first value that gets removed.
 - One side of the Queue is called the **FRONT**, where values are removed.
 - The other side is called the **REAR**, where new values are added.
 - A Queue is useful whenever things need to be processed in the same order in which they arrived.
-
- **Simple idea:**
 - A Queue works like people standing in a line. The person who arrives first gets served first.



FIFO: First In, First Out

- **FIFO means “First In, First Out.”**
- Suppose these values enter in this order:
- **10 → 20 → 30**
- Then:
- 10 entered first
- 10 will be removed first
- 20 will be removed next
- 30 will be removed after that
- **Main point:**
- Unlike a Stack, where the newest item is removed first, a Queue keeps the original arrival order.
- Queue order is based on fairness: first arrival, first service.

Queue – FIFO (First In, First Out)



Main Queue Operations

- **enqueue(value)**

- Adds a new value at the **REAR** of the Queue.
- If the Queue contains 10, 20, 30 and we enqueue 40, then 40 is added after 30.

- **dequeue()**

- Removes the value at the **FRONT**.
- If 10 is currently at the front, dequeue() removes 10 first.

- **displayAll()**

- Shows all values from the **FRONT to the REAR**.
- This helps us see the current order of elements inside the Queue.

FRONT, REAR, isEmpty(), and isFull()

- **FRONT / START**

- The FRONT tells us which element will be removed next.

- **REAR / END**

- The REAR tells us where the newest value is stored.

- **isEmpty()**

- Checks whether the Queue has no values.
- This prevents us from trying to remove something that does not exist.

- **isFull()**

- Checks whether all array positions are already being used.
- This prevents another value from being inserted when there is no available space.

Linear Queue Using an Array

A basic Queue can be stored inside a **static array**.

The array has a fixed number of positions.

The front moves forward when values are removed.

The rear moves forward when new values are added.

This works well at first, but a normal linear Queue has one important limitation.

Problem:

When the rear reaches the last array index, the Queue may appear full even if there are unused spaces at the beginning.

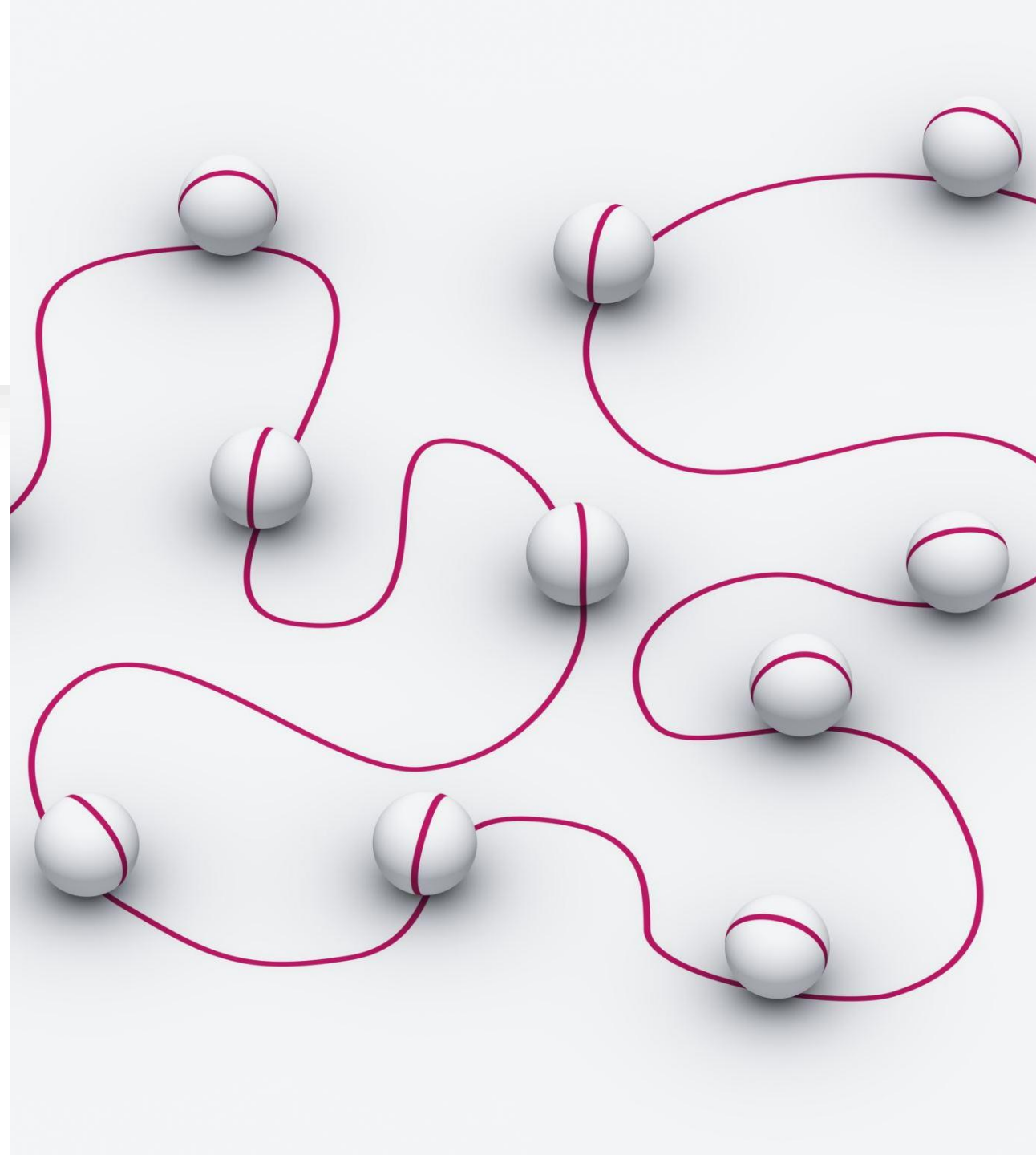
That problem is the reason we use a Circular Queue.

Why a Linear Queue Can Waste Space

- Suppose the array has 6 positions:
- 0 1 2 3 4 5
- At first:
- 10 20 30 40 50 60
- After removing 10 and 20:
- __ 30 40 50 60
- Index 0 and index 1 are empty.
- But the REAR is still at the last position.
- A simple linear Queue may not automatically reuse those empty positions.
- This creates unused space even though the array still has free locations.
- **This is exactly what Circular Queue solves.**

What Is a Circular Queue?

- A **Circular Queue** is still a Queue, so it follows FIFO.
 - The difference is how it uses the array positions.
 - When the REAR reaches the last array index, it does not stop permanently.
 - Instead, it can return to the beginning of the array and use any available empty positions.
 - This creates a circular movement through the array.
- **Why it is useful:**
- It prevents empty spaces at the beginning of the array from being wasted.



How Wrap-Around Works

Suppose the array size is 6:

0 1 2 3 4 5

If the REAR reaches index 5, the next position would normally be outside the array.

A Circular Queue avoids that by going back to:

index 0

If index 0 is free, the next value can be inserted there.

In C++, this idea can be handled using:

$(\text{end} + 1) \% \text{SIZE}$

The modulus operator % helps the index return to 0 after reaching the last array position.

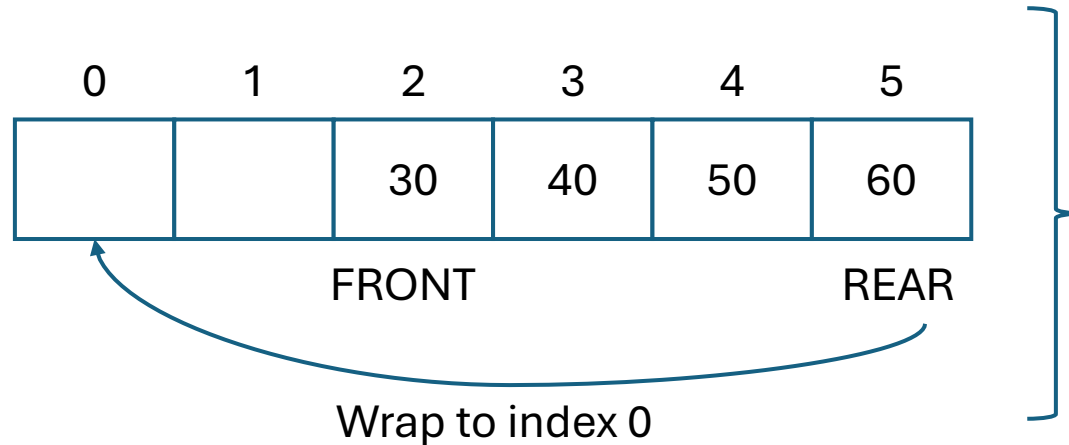
Example:

If end = 4 and SIZE = 5

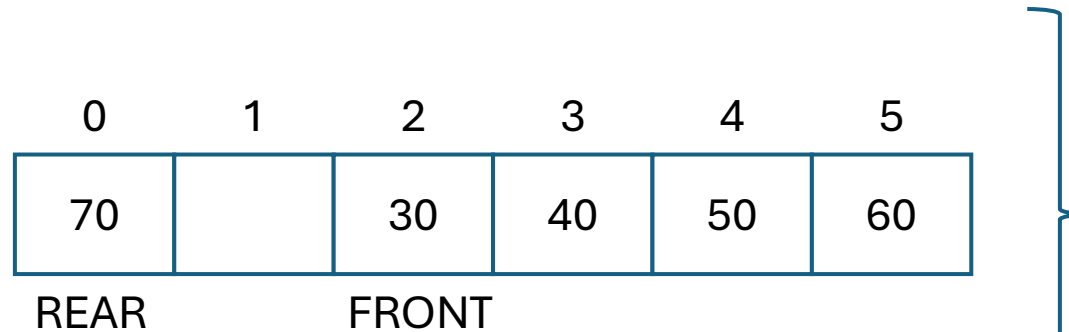
$(4 + 1) \% 5 = 0$

So REAR wraps back to index 0.

Circular Queue – Wrap-Around



Here, the front is at index 2 and the rear is at index 5. Index 0 and index 1 are empty because earlier values were removed. Since the rear has already reached the last index, the next insertion cannot continue to the right.



In a circular queue, the rear does not stop at the last index. It wraps back to index 0 and reuses the empty space. So the new value 70 is inserted at index 0, while the front stays at index 2.

A circular queue is useful because it does not waste the empty spaces that are left at the beginning of the array. Instead of stopping when the rear reaches the last index, it goes back to index 0 and uses those free positions again.

Queue Overflow and Queue Underflow

▪ Queue Overflow

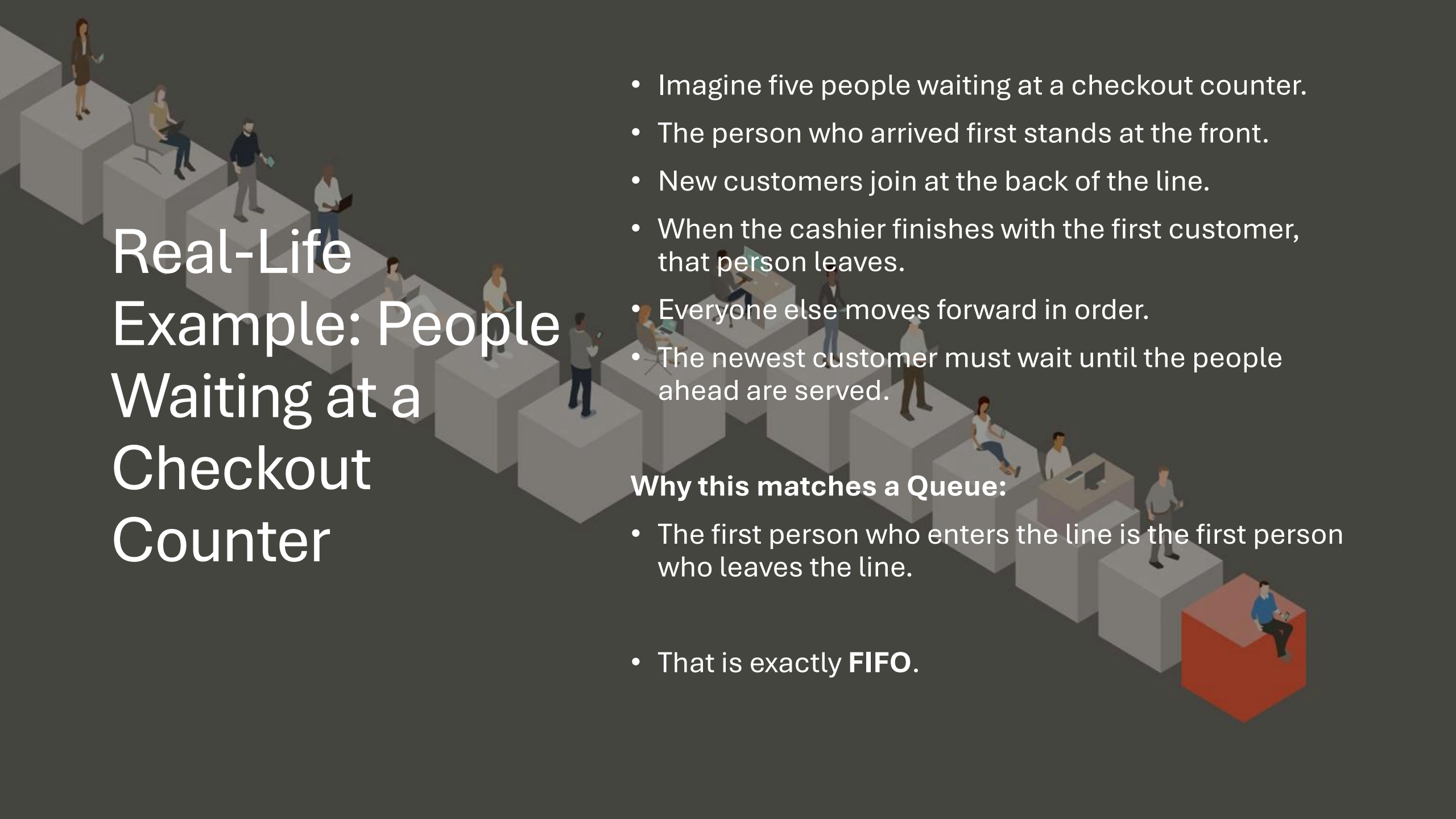
- Queue overflow happens when all available positions are already filled and we try to enqueue another value.
- In a fixed-size array, there is no extra position available.
- `isFull()` should be checked before adding a new value.

▪ Queue Underflow

- Queue underflow happens when we try to dequeue from an empty Queue.
- There is no value at the front to remove.
- `isEmpty()` should be checked before removing a value.

- **These checks make the program safer and prevent invalid operations.**





Real-Life Example: People Waiting at a Checkout Counter

- Imagine five people waiting at a checkout counter.
- The person who arrived first stands at the front.
- New customers join at the back of the line.
- When the cashier finishes with the first customer, that person leaves.
- Everyone else moves forward in order.
- The newest customer must wait until the people ahead are served.

Why this matches a Queue:

- The first person who enters the line is the first person who leaves the line.
- That is exactly **FIFO**.



Where Queues Are Used

- **Print Queue**

- When many documents are sent to a printer, they can wait in a Queue.
- The first print job sent is normally processed first.

- **Customer Service System**

- Customers can be handled in the order they arrive.
- This makes the system fair and organized.

- **Network Packet Handling**

- Data packets may wait before being processed or transmitted.
- A Queue helps maintain the arrival order.
- You can keep all three or use only two if the slide feels crowded.

Why Circular Queue Is Better Than a Simple Linear Queue

Linear Queue

FRONT and REAR move only forward

removed positions may stay unused

REAR can reach the last index even when earlier positions are empty

Circular Queue

wraps back to the beginning

reuses empty array positions

uses the fixed-size array more efficiently

works well when data is repeatedly added and removed

Main difference:

The Circular Queue makes better use of the same fixed array space.

From Theory to Live C++ Demonstration

- **Now I will demonstrate the Circular Queue using C++.**
- During the live demonstration, I will explain:
 - the static array and fixed Queue size,
 - start, end, and count,
 - how isEmpty() and isFull() work,
 - how enqueue() adds values,
 - how dequeue() removes values,
 - how displayAll() follows the circular order,
 - and how REAR wraps back to index 0.
- **After explaining the code, I will compile and run the program and show the wrap-around behaviour live.**

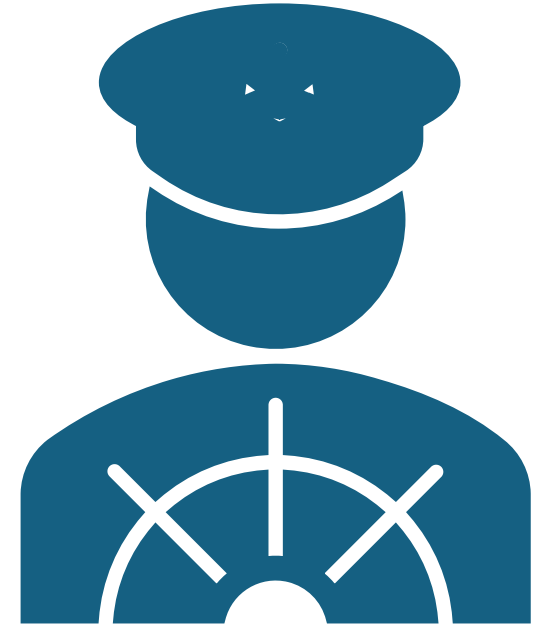


What I Learned from Queue and Circular Queue

- I learned that a Queue is not just about storing values; the order of entry and removal is very important.
- FRONT and REAR help the program keep track of where values should leave and enter.
- I understood why a normal linear Queue can waste array positions after several dequeue operations.
- The Circular Queue helped me understand how the same array space can be reused instead of creating unnecessary wasted positions.
- Implementing the Queue in C++ also helped me understand how indexes move inside an array.

References

- **Course Material – Dr. Victor Govindaswamy**
The course material helped me understand the required Queue concepts, especially FIFO, front and rear, enqueue, dequeue, the limitation of a linear Queue, and how Circular Queue wrap-around should be demonstrated.
- **GeeksforGeeks – www.geeksforgeeks.org**
I used this resource to better understand how Circular Queues reuse empty array positions and how the rear can move back to the beginning after reaching the last index.
- **W3Schools – www.w3schools.com**
I referred to W3Schools for the C++ parts of my Queue program, such as arrays, classes, member functions, loops, conditions, and user input.



Thank You

- **Thank You for Watching!**
- Today I covered:
- Queue and FIFO
- FRONT and REAR
- Enqueue and Dequeue
- Linear Queue limitation
- Circular Queue
- Wrap-around
- Overflow and Underflow
- Applications
- Live C++ demonstration
- **I hope this video made Queue and Circular Queue easier to understand. Thank you for watching and learning with me.**
- **Presented by: Akash Patel**
- **Instructor: Dr. Victor Govindaswamy**
- **Thank you for your time and attention.**